

JAVA SDE-2 INTERVIEW PREPARATION SERIES

DATA STRUCTURES AND ALGORITHMS - BOOK 14 OF 17 - STUDY STEP DSA-14

Heaps, Priority Queues, Selection, and Top-K

Heap Invariants, Comparator Safety, K-Way Processing, and Partial
Ordering

BY

Vinay Reddy Kalluri

Use heap order when only the next extremum matters, compare full sorting with selection, and write
stable, overflow-safe Java priority logic.

HEAPS | PRIORITYQUEUE | TOP-K | SELECTION | COMPARATORS | MERGING

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Updated 2026-08-02

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

60-second entry rule: If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to [DSA 13 - Trees, BSTs, and Tries](#). If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

This volume distinguishes heap order from global sorting and develops priority-based interview patterns without hiding comparator or mutation risks.

Readiness map

Decision	Evidence
START HERE	Only the next minimum or maximum is needed; The best k items must be retained; Several sorted streams must be merged; Scheduling decisions are priority-driven.
PREPARE FIRST	JAVA-01 and DSA-01 through DSA-13; Arrays, trees, sorting, and comparators.
MOVE ON WHEN	Explain array-backed heap invariants; Use PriorityQueue safely; Choose between sorting, selection, and a bounded heap; Avoid mutable-priority and comparator failures.

Choose the route that fits today

- 1 **Learning:** read in order, type the representative Java examples, and complete each dry run.
- 2 **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

DSA Segment Position

DSA 13 - Trees, BSTs, and Tries	YOU ARE HERE DSA 14 - Heaps and Priority Queues	DSA 15 - Graphs
---------------------------------	---	-----------------

A note from Vinay

A heap gives fast access to one extreme, not a sorted collection. Keep the heap invariant separate from the problem's selection invariant, especially when ties and custom comparators appear.

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION	
Start Here - Your Route Through This Volume	2
Part I - Core Learning	4
Chapter 1 - Heap Foundations: Partial Order Before Priority Patterns	4
Chapter 2 - SDE-2 Heaps, Priority Queues, Selection, and Top-K Patterns	8
Chapter 3 - Essential Heap Pattern Clinics	25
Chapter 4 - Realistic Heap and Priority-Queue Interview Rounds	29
Chapter 5 - Binary Heap and Quickselect Internals	32
Chapter 6 - Indexed Heaps, Decrease-Key, and Lazy Deletion	38
Chapter 7 - Heaps and Priority Queues Practice Lab	47
Chapter 8 - Heaps and Priority Queues Practice Lab Solutions	49
Chapter 9 - Executable Heap and Selection Checks	51
Part II - Practice and Handoff	60

Part I - Core Learning**SDE-2 CORE CHAPTER**

Chapter 1 - Heap Foundations: Partial Order Before Priority Patterns

A heap is useful when the next minimum or maximum matters more than complete sorted order. A binary min-heap guarantees that every parent is no greater than its children. It does not guarantee that siblings or entire levels are sorted.

Complete-tree shape and array indexes

A binary heap uses a complete binary-tree shape, which packs naturally into an array.

TEXT

array indexes:

```
      0
     / \
    1   2
   / \ / \
  3  4 5  6
```

$\text{left}(i) = 2*i + 1$
 $\text{right}(i) = 2*i + 2$
 $\text{parent}(i) = (i - 1) / 2$ for $i > 0$

Use bounds checks before calculating or accessing child indexes. For extremely large abstract indexes, $2*i + 1$ can overflow; real Java arrays are already bounded, but production heap-like structures should use wider arithmetic when necessary.

WHY IT WORKS | The min-heap invariant

For every index $i > 0$:

TEXT

```
heap[parent(i)] <= heap[i]
```

Therefore index 0 is globally minimal. The second-smallest item is somewhere among its children, not necessarily index 1 after arbitrary operations.

Insert: sift up

Append the new value at the end to preserve complete shape. While it violates the parent relation, swap it upward.

JAVA

```
static void siftUp(List<Integer> heap, int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (heap.get(parent) <= heap.get(index)) {
            return;
        }
        Collections.swap(heap, parent, index);
        index = parent;
    }
}
```

The height is $O(\log n)$, so insertion is $O(\log n)$.

Remove minimum: replace and sift down

Move the last value to the root, shrink the heap, and repeatedly swap with the smaller child while the invariant is violated. Choosing the smaller child is essential; swapping with an arbitrary child can leave another violation.

Why bottom-up heap construction is $O(n)$

Calling insert n times gives $O(n \log n)$. Bottom-up heapify sifts down internal nodes from the last parent to root. Most nodes are near the leaves and travel zero or one level; few nodes travel far. Summing work by height produces $O(n)$, not $O(n \log n)$.

Java PriorityQueue

JAVA

```
PriorityQueue<Integer> minimums = new PriorityQueue<>();
minimums.add(7);
minimums.add(2);
minimums.add(5);
System.out.println(minimums.peek()); // 2
System.out.println(minimums.remove()); // 2
```

For a max-heap:

JAVA

```
PriorityQueue<Integer> maximums =
    new PriorityQueue<>(Comparator.reverseOrder());
```

Important contracts:

- `peek/poll` return null on empty; `element/remove` throw;

- null elements are not permitted;
- iteration order is not sorted order;
- arbitrary `remove(object)` may be linear;
- mutating an enqueued object's priority field does not automatically reposition it.

Comparator safety

Do not subtract:

JAVA

```
// Can overflow and violate comparator ordering
(a, b) -> a.priority - b.priority
```

Use:

JAVA

```
Comparator<Job> order = Comparator
    .comparingInt(Job::priority)
    .thenComparingLong(Job::sequence);
```

Tie-breakers make output deterministic and can preserve arrival policy. The comparator must be consistent enough for a stable ordering relation; mutable comparison fields are dangerous after insertion.

Recognize bounded heaps

To retain the k largest values, use a min-heap of size at most k . The root is the weakest retained candidate. When a larger value arrives, replace the root.

TEXT

```
stream item -> compare with weakest top-k item -> discard or replace
```

This costs $O(n \log k)$ time and $O(k)$ space, often better than sorting all n values when k is small.

For k smallest values, reverse the policy: retain a max-heap of size k .

K-way merge intuition

For k sorted sources, keep only the next unconsumed item from each source in a min-heap. Polling chooses the next global output; then insert the following item from the same source. If N total items are emitted, time is $O(N \log k)$, auxiliary heap space $O(k)$, excluding output.

TRY IT | Foundation checkpoint

- 1 What order does a min-heap guarantee and what does it not guarantee?
- 2 Why does top-k largest use a min-heap?
- 3 Why is comparator subtraction unsafe?
- 4 Why is heap iteration not sorted output?
- 5 Explain bottom-up heapify without saying each node costs $O(\log n)$.

SDE-2 CORE CHAPTER

Chapter 2 - SDE-2 Heaps, Priority Queues, Selection, and Top-K Patterns

Why this chapter matters

A heap answers one focused question efficiently: which current item has the highest priority? That makes it the right structure for bounded top-k state, merging sorted sources, streaming medians, and event scheduling. It is not a fully sorted collection, a search tree, or a general indexing structure.

At SDE-2 level, "use a priority queue" is only the beginning. A complete answer defines the comparator, proves the heap or bounded-candidate invariant, handles ties and overflow, accounts for retained state, and explains deletion, streaming, and alternative selection strategies.

Learning objectives

After completing this chapter, you should be able to:

- derive parent and child indexes and state the heap-order and shape invariants;
- use Java `PriorityQueue` without comparator, mutation, or null mistakes;
- build a heap in linear time with bottom-up heapify;
- solve kth-largest and top-k-frequency problems with bounded heaps;
- merge k sorted sources while storing only one frontier item per source;
- maintain a streaming median with two balanced heaps;
- model meeting rooms and event simulation as scheduling heaps;
- explain lazy deletion when arbitrary removal is otherwise linear;
- compare a heap with sorting, counting, and quickselect; and
- discuss capacity, determinism, concurrency, backpressure, and numeric safety.

CHOOSE IT | Recognition and decision map

Signal	Candidate technique	Retained state
repeatedly remove current minimum or maximum	priority queue	all active candidates
kth largest, stream or one pass	min-heap of size k	best k values
top k by frequency	frequency map plus bounded heap	all counts, then k candidates
merge k sorted sources	min-heap frontier	next item from each nonempty source
median after every insertion	max-heap lower half plus min-heap upper half	every value, split by rank
minimum simultaneous resources	min-heap of finishing times	active resource end times
one k th element in a mutable array	quickselect	in-place partition state
complete sorted output	sorting	entire input
dense, small priority range	bucket/counting structure	counts per priority

A heap is compelling when only an extreme matters after each update. If every result must be sorted, sorting once is often simpler. If values come from a tiny bounded domain, a counting array may beat a node-heavy queue.

WHY IT WORKS | Heap model and invariants

A binary heap is a complete binary tree stored level by level in an array. Completeness supplies the shape invariant: every level except possibly the last is full, and the last fills from left to right. For zero-based index i :

TEXT

```
parent(i) = (i - 1) / 2, for i > 0
left(i)   = 2 * i + 1
right(i)  = 2 * i + 2
```

In a min-heap, every parent is no greater than either child. The root is therefore a global minimum. Siblings and separate subtrees have no ordering relationship. Iterating a Java priority queue does not produce sorted order.

Insertion appends at the complete-tree frontier, then sifts upward while heap order is violated. Removing the root moves the last item to index zero, reduces the shape, then sifts downward toward the better child. Height is $O(\log n)$, so insertion and root removal are $O(\log n)$; root inspection is $O(1)$.

The sift-down invariant is that only the current node may violate heap order; both child subtrees are already heaps. Swapping with the larger child for a max-heap repairs the old level and moves the possible violation downward. The remaining height strictly decreases.

Java PriorityQueue contracts

`PriorityQueue<E>` is a min-priority queue according to natural order or a supplied comparator. A max-heap uses a reversed comparator:

JAVA

```
PriorityQueue<Integer> maximums =
    new PriorityQueue<>(Comparator.reverseOrder());
```

Use `Integer.compare(a, b)` or comparator factories, not `a - b`, because subtraction can overflow and reverse ordering. Define every tie needed for deterministic output. A comparator must be antisymmetric, transitive, and consistent enough to establish an ordering; a broken comparator can produce inexplicable queue behavior.

The queue rejects null. `peek()` and `poll()` return null when empty, while `element()` and `remove()` throw. Choose deliberately. Removing an arbitrary object with `remove(Object)` is linear because the heap does not index its internal position.

Never mutate fields used by the comparator while an object is inside the queue. The queue does not notice the priority changed and does not relocate the object. Insert an immutable snapshot or remove and reinsert through a controlled index-aware design.

Pattern 1: bottom-up heapify

To turn an arbitrary array into a max-heap, every leaf is already a one-node heap. Starting at the last parent $n / 2 - 1$, sift each node down toward index zero.

Invariant before processing index `i`: every subtree rooted at an index greater than `i` is already a max-heap. Sifting `i` repairs its subtree because its child subtrees satisfy the invariant. At termination, index zero roots a heap spanning the array.

Although an individual sift can take $O(\log n)$, bottom-up heapify is $O(n)$. Most nodes are near the leaves and move at most one or two levels. Summing `nodesAtHeight(h) * h` over heights is bounded by a constant multiple of `n`.

TRACE IT | Dry run

For `[3, 1, 6, 5, 2, 4]`, the last parent is index 2. Index 2 already dominates child 4. At index 1, swap 1 with child 5: `[3, 5, 6, 1, 2, 4]`. At index 0, swap 3 with 6, then with 4: `[6, 5, 4, 1, 2, 3]`. Every parent now dominates its children.

Heapify is useful for heap sort or an internal primitive heap. Constructing Java `PriorityQueue` from a collection also performs implementation-managed heap construction; prefer the library unless the interview asks for derivation or primitive storage matters.

Pattern 2: kth largest with a bounded min-heap

Keep a min-heap containing the largest k values seen. For each value:

- 1 add it;
- 2 if size exceeds k , remove the minimum.

Invariant after each prefix: the heap contains exactly the largest $\min(k, \text{processed})$ values from that prefix, including duplicates according to occurrence. The root is the smallest among those retained values. After all values, it is the k th largest.

For $[3, 2, 1, 5, 6, 4]$, $k=2$, retained heaps by sorted content are $[3]$, $[2, 3]$, $[2, 3]$, $[3, 5]$, $[5, 6]$, $[5, 6]$; the root 5 is second largest.

Time is $O(n \log k)$, space $O(k)$. Validate $1 \leq k \leq n$. Sorting takes $O(n \log n)$ and may be preferable when all ranks are later needed. Quickselect has expected $O(n)$ for one offline rank but mutates the input and has a quadratic worst case without stronger pivot selection.

Pattern 3: top k frequent values

First count every value in a hash map. Then keep a size- k min-heap of frequency records. The root represents the least desirable retained record, so it is the first evicted when a stronger candidate arrives.

Tie policy is part of the answer. The implementation defines higher frequency as better and, for equal frequency, smaller numeric value as better. The eviction comparator therefore treats lower frequency as worse and, on ties, larger value as worse. Final results are sorted into presentation order.

For values $[1, 1, 1, 2, 2, 3]$, $k=2$, counts are $\{1=3, 2=2, 3=1\}$. The bounded heap retains 1 and 2. Expected time is $O(n + d \log k)$ for d distinct values, and space is $O(d + k)$. Bucket-by-frequency can reach $O(n)$ time but allocates buckets indexed up to n ; sorting the d entries costs $O(d \log d)$ and may be simpler for large k .

Pattern 4: k -way merge

Each sorted source exposes only its next unconsumed value. Put one cursor per nonempty source in a min-heap. Repeatedly remove the smallest cursor, emit its value, and insert the next value from that same source.

Invariant: the heap contains exactly the first unconsumed value of each nonexhausted source. Because each source is sorted, the smallest unconsumed value globally must be among those frontier items. Emitting the heap root is therefore safe.

For $[1, 4, 9]$, $[2, 2, 8]$, and $[3, 7]$, the initial frontier is 1,2,3. Emitting 1 exposes 4; emitting the first 2 exposes another 2; and so on. Output is $[1, 2, 2, 3, 4, 7, 8, 9]$.

For N total elements and k sources, time is $O(N \log k)$ and heap space is $O(k)$, excluding output. A streaming version need not retain the entire result. It should close sources, handle one failed source according to policy, and apply backpressure when consumers are slow.

Pattern 5: streaming median with two heaps

Maintain:

- **lower**: a max-heap containing the lower half;
- **upper**: a min-heap containing the upper half.

Invariants:

- 1 every value in **lower** is no greater than every value in **upper**;
- 2 their sizes differ by at most one; and
- 3 **lower** has the extra item when the total count is odd.

Insert into **lower** when the value is at most its root, otherwise into **upper**. Rebalance by moving a root across if sizes violate the rule. Median is **lower.peek()** for odd count and the average of both roots for even count.

For stream **5, 2, 10, 4**, heap contents by sorted meaning become: lower **[5]**; lower **[2]**, upper **[5]**; lower **[5, 2]**, upper **[10]**; lower **[4, 2]**, upper **[5, 10]**. Medians are **5, 3.5, 5, 4.5**.

Each insertion costs $O(\log n)$ and median lookup $O(1)$; space is $O(n)$. Compute the even median as $((\text{long}) a + b) / 2.0$ to avoid integer overflow.

Pattern 6: scheduling with finishing-time heaps

For minimum meeting rooms, sort intervals by start. Keep a min-heap of end times for meetings currently occupying rooms. Before adding a meeting, remove every end time no greater than its start under half-open interval semantics **[start, end)**. A zero-length interval **[t, t)** is empty and consumes no room, so skip it. Add the end of each nonempty meeting and record the maximum heap size.

Invariant before adding the next sorted meeting: the heap contains exactly the end times of earlier meetings overlapping its start. Its size is the resources currently busy. Adding the meeting gives current concurrent demand.

For **[0, 30), [5, 10), [15, 20)**, maximum size is 2. The meeting ending at 10 is removed before 15 begins, so its room is reused.

Time is $O(n \log n)$ for sorting and heap operations; space is $O(n)$. If only room count is needed, sorting separate start and end arrays enables a two-pointer sweep. A heap is more natural when assigning room identifiers or processing an online event stream.

The same model supports CPU/job scheduling, timers, and simulation: priority is the next event time. Real schedulers add cancellation, priority classes, fairness, and starvation prevention, which a single comparator does not solve automatically.

Lazy deletion for arbitrary removal

Priority queues efficiently remove only the root. Sliding-window median and cancellable jobs may need to remove an item that is buried inside. Calling `remove(Object)` for every expiry can make the algorithm quadratic.

Lazy deletion separates logical deletion from physical removal:

- 1 give items stable identities or use value counts when duplicates are interchangeable;
- 2 record cancelled/expired identities in a delayed-deletion map;
- 3 decrement logical size immediately;
- 4 whenever inspecting a heap root, repeatedly discard roots marked delayed and decrement their delayed counts.

Invariant: logical sizes exclude delayed items even if their nodes remain physically present; after pruning, a visible root is live. This supports amortized $O(\log n)$ updates because each stale node is physically removed once.

The design is subtle with duplicate values and two heaps. Value-only deletion must know which logical side loses size; stable `(value, id)` entries avoid ambiguity. Delayed maps can retain metadata if pruning never reaches buried cancelled items, so periodic rebuild or queue compaction may be needed in long-running systems.

Quickselect boundary

Quickselect partitions around a pivot: values less than the pivot move to one side and greater values to the other. Only the partition containing the target rank is explored. Expected time is $O(n)$, auxiliary stack space is expected $O(\log n)$ or $O(1)$ iteratively, and the input is mutated. Poor pivots can cause $O(n^2)$ time; randomized pivots make that unlikely but not impossible.

Choose quickselect for one or a few offline ranks when mutation is acceptable. Choose a bounded heap for streaming data, immutable input, small k , or ongoing updates. Choose sorting when ordered output is required. A deterministic linear selection algorithm exists, but its constants and implementation complexity rarely make it the first production choice.

Testing heap algorithms beyond examples

Example assertions catch boundary mistakes, but heap code benefits from properties that hold across generated inputs. After heapify, check every parent against both existing children rather than comparing only the root. Also verify the output array is a permutation of the input; heap order alone would not reveal a duplicated or lost value. For repeated removal, emitted values must be monotone and their multiset must equal the input multiset.

For k th selection, compare random small cases with a sorted reference and test every valid k , including duplicates and `Integer.MIN_VALUE/MAX_VALUE`. For top- k frequency, independently count the output and ensure no excluded item is better than a retained item under the complete tie comparator. For k -way merge, verify global sortedness, output count, and exact multiplicity from every source. Empty sources, one source, all-equal values, and highly uneven source sizes expose frontier errors.

The two-heap median has three useful properties after every insertion: size difference is at most one, lower root is no greater than upper root, and the reported value matches a sorted reference for a short test stream. Test even medians at integer extremes to catch overflow. Scheduling tests should include zero-duration intervals, many simultaneous starts, and exact end/start ties under the chosen half-open contract.

For a custom heap, randomized operation sequences can be checked against [PriorityQueue](#). Record a seed when a test fails so the sequence is reproducible. Avoid tests that assume [PriorityQueue](#) iteration order; compare repeated polls from a copy when sorted priority order is required. These properties turn an implementation invariant into an executable release gate.

Runnable Java 21 reference implementation

Run with `java -ea HeapSelectionPatterns`.

JAVA (continued 1/7)

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.PriorityQueue;

public final class HeapSelectionPatterns {
    private HeapSelectionPatterns() {}

    private record Frequency(int value, int count) {}

    private record Cursor(int value, int source, int index) {}

    public static void heapifyMax(int[] values) {
        requireArray(values);
        for (int i = values.length / 2 - 1; i >= 0; i--) {
            siftDownMax(values, i, values.length);
        }
    }

    public static boolean isMaxHeap(int[] values) {
        requireArray(values);
        for (int parent = 0; parent < values.length / 2; parent++) {
            int left = 2 * parent + 1;
            int right = left + 1;
            if (values[parent] < values[left]
                || right < values.length && values[parent] < values[right]) {
                return false;
            }
        }
    }
}
```

JAVA (continued 2/7)

```
    }  
    }  
    return true;  
}  
  
public static int kthLargest(int[] values, int k) {  
    requireArray(values);  
    if (k < 1 || k > values.length) {  
        throw new IllegalArgumentException("k must be in 1..n");  
    }  
    PriorityQueue<Integer> retained = new PriorityQueue<>();  
    for (int value : values) {  
        retained.add(value);  
        if (retained.size() > k) {  
            retained.remove();  
        }  
    }  
    return retained.element();  
}  
  
public static List<Integer> topKFrequent(int[] values, int k) {  
    requireArray(values);  
    Map<Integer, Integer> counts = new HashMap<>();  
    for (int value : values) {  
        counts.merge(value, 1, Integer::sum);  
    }  
    if (k < 0 || k > counts.size()) {  
        throw new IllegalArgumentException("k must be in 0..distinct");  
    }  
    Comparator<Frequency> worstFirst = Comparator  
        .comparingInt(Frequency::count)  
        .thenComparing(Comparator.comparingInt(Frequency::value).reversed());  
    PriorityQueue<Frequency> best = new PriorityQueue<>(worstFirst);
```

JAVA (continued 3/7)

```
    for (Map.Entry<Integer, Integer> entry : counts.entrySet()) {
        best.add(new Frequency(entry.getKey(), entry.getValue()));
        if (best.size() > k) {
            best.remove();
        }
    }
    List<Frequency> ordered = new ArrayList<>(best);
    ordered.sort(Comparator.comparingInt(Frequency::count).reversed()
        .thenComparingInt(Frequency::value));
    return ordered.stream().map(Frequency::value).toList();
}

public static List<Integer> mergeSorted(int[][] sources) {
    if (sources == null) {
        throw new IllegalArgumentException("sources must not be null");
    }
    PriorityQueue<Cursor> frontier = new PriorityQueue<>(
        Comparator.comparingInt(Cursor::value)
            .thenComparingInt(Cursor::source)
            .thenComparingInt(Cursor::index));

    int total = 0;
    for (int source = 0; source < sources.length; source++) {
        int[] row = sources[source];
        if (row == null) {
            throw new IllegalArgumentException("source must not be null");
        }
        total = Math.addExact(total, row.length);
        for (int i = 1; i < row.length; i++) {
            if (row[i] < row[i - 1]) {
                throw new IllegalArgumentException("sources must be sorted");
            }
        }
        if (row.length > 0) {
```

JAVA (continued 4/7)

```

        frontier.add(new Cursor(row[0], source, 0));
    }
}
List<Integer> result = new ArrayList<>(total);
while (!frontier.isEmpty()) {
    Cursor next = frontier.remove();
    result.add(next.value());
    int following = next.index() + 1;
    if (following < sources[next.source()].length) {
        frontier.add(new Cursor(sources[next.source()][following],
                                next.source(), following));
    }
}
return result;
}

public static int minimumMeetingRooms(int[][] intervals) {
    if (intervals == null) {
        throw new IllegalArgumentException("intervals must not be null");
    }
    int[][] copy = new int[intervals.length][2];
    for (int i = 0; i < intervals.length; i++) {
        if (intervals[i] == null || intervals[i].length != 2
            || intervals[i][0] > intervals[i][1]) {
            throw new IllegalArgumentException("invalid interval");
        }
        copy[i] = intervals[i].clone();
    }
    Arrays.sort(copy, Comparator.comparingInt((int[] interval) -> interval[0])
        .thenComparingInt(interval -> interval[1]));
    PriorityQueue<Integer> activeEnds = new PriorityQueue<>();
    int maximum = 0;
    for (int[] interval : copy) {

```

JAVA (continued 5/7)

```
        if (interval[0] == interval[1]) {
            continue;
        }
        while (!activeEnds.isEmpty() && activeEnds.element() <= interval[0]) {
            activeEnds.remove();
        }
        activeEnds.add(interval[1]);
        maximum = Math.max(maximum, activeEnds.size());
    }
    return maximum;
}

public static final class RunningMedian {
    private final PriorityQueue<Integer> lower =
        new PriorityQueue<>(Comparator.reverseOrder());
    private final PriorityQueue<Integer> upper = new PriorityQueue<>();

    public void add(int value) {
        if (lower.isEmpty() || value <= lower.element()) {
            lower.add(value);
        } else {
            upper.add(value);
        }
        if (lower.size() < upper.size()) {
            lower.add(upper.remove());
        } else if (lower.size() > upper.size() + 1) {
            upper.add(lower.remove());
        }
    }

    public double median() {
        if (lower.isEmpty()) {
```

JAVA (continued 6/7)

```
        throw new IllegalStateException("no values");
    }
    if (lower.size() > upper.size()) {
        return lower.element();
    }
    return ((long) lower.element() + upper.element()) / 2.0;
}

public int size() {
    return lower.size() + upper.size();
}
}

private static void siftDownMax(int[] values, int root, int size) {
    int current = root;
    while (true) {
        int left = 2 * current + 1;
        if (left >= size) {
            return;
        }
        int right = left + 1;
        int larger = right < size && values[right] > values[left] ? right : left;
        if (values[current] >= values[larger]) {
            return;
        }
        int temporary = values[current];
        values[current] = values[larger];
        values[larger] = temporary;
        current = larger;
    }
}
```

JAVA (continued 7/7)

```
private static void requireArray(int[] values) {
    if (values == null) {
        throw new IllegalArgumentException("values must not be null");
    }
}

public static void main(String[] args) {
    int[] heap = {3, 1, 6, 5, 2, 4};
    heapifyMax(heap);
    assert isMaxHeap(heap);
    assert heap[0] == 6;
    assert kthLargest(new int[] {3, 2, 1, 5, 6, 4}, 2) == 5;
    assert topKFrequent(new int[] {1, 1, 1, 2, 2, 3}, 2)
        .equals(List.of(1, 2));

    assert mergeSorted(new int[][] {{1, 4, 9}, {2, 2, 8}, {3, 7}})
        .equals(List.of(1, 2, 2, 3, 4, 7, 8, 9));
    assert minimumMeetingRooms(new int[][] {{0, 30}, {5, 10}, {15, 20}}) == 2;
    assert minimumMeetingRooms(new int[][] {{5, 5}}) == 0;

    RunningMedian median = new RunningMedian();
    median.add(5);
    assert median.median() == 5.0;
    median.add(2);
    assert median.median() == 3.5;
    median.add(10);
    assert median.median() == 5.0;
    median.add(4);
    assert median.median() == 4.5;
    assert median.size() == 4;
}
```

Complexity and failure-mode table

Operation/pattern	Time	Space	Main risk
bottom-up heapify	$O(n)$	$O(1)$	treating every node as height $\log n$
kth largest	$O(n \log k)$	$O(k)$	wrong min/max heap orientation
top-k frequency	expected $O(n + d \log k)$	$O(d + k)$	undefined tie order
k-way merge	$O(N \log k)$	$O(k)$ plus output	storing all elements in the heap
streaming median insertion	$O(\log n)$	$O(n)$	order or balance invariant broken
meeting rooms	$O(n \log n)$	$O(n)$	endpoint reuse semantics undefined
lazy deletion update	amortized $O(\log n)$	can exceed live size	stale metadata never compacted
quickselect	expected $O(n)$, worst $O(n^2)$	depends on implementation	mutation and pivot worst case

WATCH OUT | Edge cases and common mistakes

- 1 **Wrong heap orientation.** Kth largest retains a min-heap so the weakest retained value is removable.
- 2 **Overflowing comparator.** Never implement priority with subtraction.
- 3 **Mutable priority field.** Reinsert an immutable updated entry.
- 4 **Assuming iteration is sorted.** Only repeated `poll` produces priority order and it destroys the queue.
- 5 **Invalid k.** Define $k=0$ only for top-k output; kth rank requires $1 \leq k \leq n$.
- 6 **Dropping duplicates.** Rank by occurrences unless the question explicitly says distinct values.
- 7 **Median overflow.** Widen before adding two middle integers.
- 8 **One heap unbalanced.** State which side owns the odd extra value.
- 9 **K-way merge without sorted validation.** The frontier proof depends on every source order.
- 10 **Closed versus half-open meetings.** Decide whether an end at time t frees a room for a start at t ; under $[start, end)$, $[t, t)$ is empty and must not allocate a room.
- 11 **Arbitrary queue removal assumed logarithmic.** Standard `remove(Object)` searches linearly.
- 12 **Ignoring output space.** Merging into a list stores all N results even though the frontier is $O(k)$.

SDE-2 production follow-ups

- **Backpressure:** a k-way streaming merge must not read unbounded data ahead of a slow consumer.
- **Failure policy:** decide whether one failed source aborts the merge, is retried, or yields a partial result with provenance.
- **Concurrency:** `PriorityQueue` is not thread-safe. `PriorityBlockingQueue` is concurrent but unbounded by default and does not supply every scheduling policy.
- **Capacity:** bounded top-k heaps naturally cap candidate memory, but the preceding frequency map may still have unbounded distinct cardinality.
- **Determinism:** include stable tie-break fields such as sequence number when equal priority must preserve arrival order.
- **Clock semantics:** schedulers need monotonic time for durations, wall time for calendar meaning, and a policy for clock changes.
- **Cancellation:** stable job IDs plus lazy deletion or an indexed heap avoid repeated linear scans.
- **Metrics:** track live size, physical size, delayed-delete count, oldest wait, processing lag, and rejected jobs.
- **Numeric types:** weights, timestamps, sums, and counts often require `long`; median averaging may require `double` or an exact rational/decimal contract.
- **Library choice:** prefer Java's maintained queue for object priorities. A custom primitive heap is justified by measured allocation or API needs and deserves dedicated property tests.

TRY IT | Exercises with model checkpoints

Exercise 1: kth smallest in sorted rows

Find the kth smallest value across sorted arrays without flattening.

Model checkpoints: use the k-way frontier; pop exactly k times; validate total count and k; complexity $O(k \log \text{rows})$ and space $O(\text{rows})$; duplicates count as separate occurrences.

Exercise 2: sliding-window median

Return the median for every width-k window.

Model checkpoints: two heaps plus delayed deletion; identify entries by `(value, index)` to disambiguate duplicates; maintain logical sizes; prune before reading roots; $O(n \log k)$ expected/amortized and $O(k)$ live state, with possible stale physical entries.

Exercise 3: assign meeting-room IDs

Return a room number for each meeting.

Model checkpoints: active heap ordered by end time and a second min-heap of free room IDs; retain original interval index; define tie reuse; output assignments while maximum allocated ID gives room count.

Exercise 4: merge infinite iterators

Merge sorted iterators lazily.

Model checkpoints: one frontier element per source; do not call `next` without `hasNext`; surface source failures; close resources; consumer cancellation must stop producers; returned iterator owns the heap.

Exercise 5: top-k with changing scores

Items receive score updates and queries request current top k.

Model checkpoints: inserting new snapshots creates stale entries; version each item and lazily discard old roots, or use an indexed heap; cap stale growth; define tie order and consistency during concurrent updates.

Exercise 6: choose selection strategy

Compare sorting, quickselect, and a bounded heap for 100 million values and `k=20`.

Model checkpoints: streaming and memory favor the heap; offline mutable storage may favor quickselect; sorted downstream consumption favors sorting; include I/O cost, parallelism, worst-case requirement, and repeated-query plans.

Interview answer checklist

- ☐ I stated the heap orientation and comparator tie order.
- ☐ I named the retained-candidate or frontier invariant.
- ☐ I used expected/amortized wording where appropriate.
- ☐ I counted duplicates according to the rank contract.
- ☐ I widened comparator and median arithmetic safely.
- ☐ I separated live state, delayed stale state, and output space.
- ☐ I defined empty, k-boundary, and interval-endpoint behavior.
- ☐ I can compare heap, sorting, buckets, and quickselect.
- ☐ I can explain cancellation, backpressure, determinism, and concurrency.

RECAP | Summary

Heaps maintain an extreme under updates, not a complete order. Bottom-up heapify derives a valid tree in linear time. A bounded min-heap retains the largest k values; a map plus heap retains top frequencies; a frontier heap merges sorted sources; two ordered heaps expose a streaming median; and end-time heaps model active scheduled work. Lazy deletion compensates for arbitrary-removal limits, while quickselect offers a different offline rank trade-off. A production-grade answer makes the comparator, invariant, tie policy, retained memory, stale state, and operational behavior explicit.

SDE-2 CORE CHAPTER

Chapter 3 - Essential Heap Pattern Clinics

Bounded selection and k-way merge are the main heap families. K closest points applies bounded selection with safe comparison, while smallest range covering k lists extends k-way merge by tracking both frontier extremes.

Clinic 1: k closest points with a bounded max-heap

To retain the k best points seen so far, keep the worst retained point at the heap root. For closest points, that means a max-heap by squared distance. After inserting a point, remove the root when size exceeds k.

Squared distance uses `long`:

TEXT

```
(long) x * x + (long) y * y
```

Do not use square roots, and do not subtract distances inside a comparator. Deterministic coordinate tie-breakers make tests and APIs repeatable.

This approach costs $O(n \log k)$ time and $O(k)$ space. Sorting all points costs $O(n \log n)$. Quickselect has expected $O(n)$ time but mutates or copies input and needs a worst-case policy.

Clinic 2: smallest range covering one value from every sorted list

Put the first value from each list into a min-heap and track the maximum value currently represented. The heap root and current maximum define a range containing one frontier value from every list.

After evaluating the range, advance only the list that supplied the minimum. Advancing another list cannot increase the minimum, so it cannot improve the left boundary. Stop when any list is exhausted because no later frontier can cover all lists.

For:

TEXT

```
[4, 10, 15, 24, 26]
[0, 9, 12, 20]
[5, 18, 22, 30]
```

the best range is `[20, 24]`.

There are N values across k lists. Each enters and leaves the heap at most once, giving $O(N \log k)$ time and $O(k)$ heap space.

Runnable Java 21 clinic

JAVA (continued 1/3)

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Objects;
import java.util.PriorityQueue;

public final class HeapCoverageClinic {
    private HeapCoverageClinic() {}

    public record Point(int x, int y) {
        public long squaredDistance() {
            return (long) x * x + (long) y * y;
        }
    }

    private record Entry(int value, int listIndex, int valueIndex) {}

    public static List<Point> kClosest(List<Point> points, int k) {
        Objects.requireNonNull(points, "points");
        if (k < 0 || k > points.size()) {
            throw new IllegalArgumentException("k is outside the input");
        }
        Comparator<Point> worstFirst = (first, second) -> {
            int byDistance = Long.compare(second.squaredDistance(), first.squaredDistance());
            if (byDistance != 0) {
                return byDistance;
            }
            int byX = Integer.compare(second.x(), first.x());
            return byX != 0 ? byX : Integer.compare(second.y(), first.y());
        };
        PriorityQueue<Point> retained = new PriorityQueue<>(worstFirst);
        for (Point point : points) {
```

JAVA (continued 2/3)

```
        retained.add(Objects.requireNonNull(point, "point"));
        if (retained.size() > k) {
            retained.remove();
        }
    }
    List<Point> answer = new ArrayList<>(retained);
    answer.sort(Comparator.comparingLong(Point::squaredDistance)
        .thenComparingInt(Point::x).thenComparingInt(Point::y));
    return List.copyOf(answer);
}

public static int[] smallestCoveringRange(List<List<Integer>> lists) {
    Objects.requireNonNull(lists, "lists");
    if (lists.isEmpty()) {
        throw new IllegalArgumentException("at least one list is required");
    }
    PriorityQueue<Entry> minimums = new PriorityQueue<>(
        Comparator.comparingInt(Entry::value)
            .thenComparingInt(Entry::listIndex));
    int currentMaximum = Integer.MIN_VALUE;

    for (int listIndex = 0; listIndex < lists.size(); listIndex++) {
        List<Integer> values = Objects.requireNonNull(lists.get(listIndex), "list");
        if (values.isEmpty()) {
            throw new IllegalArgumentException("every list must be nonempty");
        }
        int first = values.get(0);
        minimums.add(new Entry(first, listIndex, 0));
        currentMaximum = Math.max(currentMaximum, first);
    }

    int bestStart = minimums.element().value();
    int bestEnd = currentMaximum;
    while (minimums.size() == lists.size()) {
```

JAVA (continued 3/3)

```

        Entry minimum = minimums.remove();
        long width = (long) currentMaximum - minimum.value();
        long bestWidth = (long) bestEnd - bestStart;
        if (width < bestWidth || (width == bestWidth && minimum.value() < bestStart)) {
            bestStart = minimum.value();
            bestEnd = currentMaximum;
        }

        int nextIndex = minimum.valueIndex() + 1;
        List<Integer> source = lists.get(minimum.listIndex());
        if (nextIndex == source.size()) {
            break;
        }
        int nextValue = source.get(nextIndex);
        minimums.add(new Entry(nextValue, minimum.listIndex(), nextIndex));
        currentMaximum = Math.max(currentMaximum, nextValue);
    }
    return new int[] {bestStart, bestEnd};
}

public static void main(String[] args) {
    List<Point> closest = kClosest(
        List.of(new Point(1, 3), new Point(-2, 2), new Point(5, 8)), 2);
    assert closest.equals(List.of(new Point(-2, 2), new Point(1, 3)));

    int[] range = smallestCoveringRange(List.of(
        List.of(4, 10, 15, 24, 26),
        List.of(0, 9, 12, 20),
        List.of(5, 18, 22, 30)));
    assert range[0] == 20 && range[1] == 24;
    System.out.println("PASS essential heap clinics");
}
}

```

Expected output with assertions enabled:

TEXT

PASS essential heap clinics

Interviewer follow-up chain with model answers

Interviewer: Why use a max-heap for the k smallest distances?

Candidate: The root should be the easiest retained candidate to evict. Keeping the largest retained distance at the root makes each replacement $O(\log k)$.

Interviewer: Does iterating the heap give the answer in distance order?

Candidate: No. A priority queue guarantees only its head. If ordered output is required, I sort the k retained values or repeatedly poll a copy.

Interviewer: Why can the covering-range loop stop when one source ends?

Candidate: Every candidate range needs one value from every list. Once the list that supplied the current minimum has no successor, no later complete frontier exists.

SDE-2 CORE CHAPTER

Chapter 4 - Realistic Heap and Priority-Queue Interview Rounds

Round 1: kth largest value in a stream

Prompt

Design a class that accepts values one at a time and reports the kth largest value seen so far once at least k values exist.

Candidate design

Maintain a min-heap of the largest k values. Its root is the kth largest because every retained value is at least as large as every discarded value.

JAVA

```
static final class KthLargest {
    private final int k;
    private final PriorityQueue<Integer> largest = new PriorityQueue<>();

    KthLargest(int k) {
        if (k <= 0) throw new IllegalArgumentException("k must be positive");
        this.k = k;
    }

    OptionalInt add(int value) {
        if (largest.size() < k) {
            largest.add(value);
        } else if (value > largest.peek()) {
            largest.poll();
            largest.add(value);
        }
        return largest.size() == k ? OptionalInt.of(largest.peek()) : OptionalInt.empty();
    }
}
```

Duplicates count as separate stream observations unless the contract says kth distinct. Each add costs $O(\log k)$ when the heap changes and $O(1)$ when discarded.

Follow-up answers

Kth distinct? Add a membership set synchronized with the heap, and remove from the set when evicting. Clarify whether unbounded historical duplicates need tracking.

Distributed stream? Local top-k summaries can be merged, but ordering, duplicate identity, late events, and consistency windows need a system contract.

Round 2: merge k sorted linked lists

Prompt

Merge k sorted linked lists by reusing nodes.

Model answer

Put each non-null head in a min-heap. Poll the smallest node, append it, and enqueue its original successor.

JAVA

```
static ListNode merge(ListNode[] lists) {
    PriorityQueue<ListNode> queue = new PriorityQueue<>(
        Comparator.comparingInt(node -> node.value));
    for (ListNode head : lists) {
        if (head != null) queue.add(head);
    }
    ListNode sentinel = new ListNode(0);
    ListNode tail = sentinel;
    while (!queue.isEmpty()) {
        ListNode node = queue.poll();
        if (node.next != null) queue.add(node.next);
        tail.next = node;
        tail = node;
    }
    return sentinel.next;
}
```

If there are N nodes total, time is $O(N \log k)$, heap space $O(k)$, and relinking uses $O(1)$ extra nodes. The input lists are consumed/relinked.

Follow-up answers

Why enqueue successor before or after append? Either can work if the original successor reference remains accessible. With more aggressive detachment, save it before overwriting links.

What if lists share nodes? Destructive merge can duplicate or cycle. Require disjoint ownership or copy nodes and track identity.

Round 3: running median

Prompt

After each inserted integer, return the median.

Candidate design

Use a max-heap **lower** for the smaller half and min-heap **upper** for the larger half. Maintain:

- sizes differ by at most one;
- **lower** may contain one extra item;
- every lower value is no greater than every upper value.

JAVA

```
static final class MedianTracker {  
    private final PriorityQueue<Integer> lower =  
        new PriorityQueue<>(Comparator.reverseOrder());  
    private final PriorityQueue<Integer> upper = new PriorityQueue<>();  
  
    void add(int value) {  
        if (lower.isEmpty() || value <= lower.peek()) lower.add(value);  
        else upper.add(value);  
        if (lower.size() > upper.size() + 1) upper.add(lower.poll());  
        else if (upper.size() > lower.size()) lower.add(upper.poll());  
    }  
  
    double median() {  
        if (lower.isEmpty()) throw new IllegalStateException("no values");  
        if (lower.size() != upper.size()) return lower.peek();  
        return ((long) lower.peek() + upper.peek()) / 2.0;  
    }  
}
```

Cast before addition to prevent **int** overflow.

Follow-up answers

Sliding-window median? Arbitrary expiry is not efficient with plain **PriorityQueue**; use lazy deletion keyed by value/index, indexed heaps, or balanced multisets with counts.

Concurrency? Both heaps and their invariant must be updated atomically. A thread-safe wrapper needs one coherent synchronization boundary.

Closing answer pattern

State the retained partial order, heap direction, size bound, comparator/tie policy, mutation assumptions, per-operation and total costs, and why sorting everything is unnecessary.

SDE-2 CORE CHAPTER

Chapter 5 - Binary Heap and Quickselect Internals

A priority queue answers "which item should come out next?" It does not keep every item in sorted iteration order. A binary heap is the compact structure that usually makes that contract efficient. Quickselect solves a different contract: find one rank in a fixed collection without maintaining a stream.

This chapter builds both from first principles, then reconnects them to Java's `PriorityQueue`. Complete executable code is in `HeapInterviewChecks.java`.

WHY IT WORKS | Heap shape and order are separate invariants

A binary min-heap has two properties:

- 1 **Shape:** it is a complete binary tree-levels fill left to right.
- 2 **Order:** every parent is less than or equal to each child.

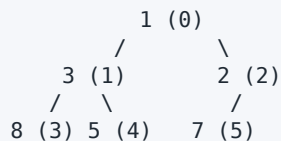
The shape permits dense array storage. For zero-based index `i`:

TEXT

```
parent      = (i - 1) / 2      when i > 0
left child = 2 * i + 1
right child = 2 * i + 2
```

TEXT

array: [1, 3, 2, 8, 5, 7]



This is a valid heap but not a sorted array. Only the root is globally minimum. A breadth-first traversal or `PriorityQueue` iterator does not promise sorted order.

Offer: append, then sift up

Appending preserves the complete-tree shape but may violate parent order. Compare the new child with its parent and swap upward until order is restored.

TEXT

```
offer 0 into [1, 3, 2, 8, 5, 7]

[1, 3, 2, 8, 5, 7, 0]  compare indexes 6 and 2
[1, 3, 0, 8, 5, 7, 2]  compare indexes 2 and 0
[0, 3, 1, 8, 5, 7, 2]  stop at root
```

The tree height is $O(\log n)$, so offer is $O(\log n)$ worst case.

The companion doubles its backing array when full. Geometric growth makes append capacity work amortized $O(1)$; sifting still makes the complete offer $O(\log n)$.

Poll: move the last value, then sift down

Removing the root leaves a hole. Move the last value to index zero, shrink logical size, then repeatedly swap with the smaller child.

TEXT

```
poll from [1, 3, 2, 8, 5, 7]

remove 1; move 7 to root: [7, 3, 2, 8, 5]
smaller child is 2:      [2, 3, 7, 8, 5]
stop: 7 has no child
```

Always choose the smaller child in a min-heap. Swapping with the left child merely because it exists can leave a smaller right child under a larger parent.

The removed array slot does not belong to logical heap state after **size** decreases. For object heaps, clear that slot so the backing array does not retain an unnecessary reference.

Bottom-up heapify is $O(n)$, not $O(n \log n)$

To heapify an existing array, every leaf is already a valid one-node heap. Start at the last parent $n / 2 - 1$ and sift each parent downward.

Although one sift can be $O(\log n)$, most nodes are near the leaves and move very little. Roughly half the nodes have height zero, one quarter height one, one eighth height two, and so on:

TEXT

```
 $n/4 * 1 + n/8 * 2 + n/16 * 3 + \dots = O(n)$ 
```

Building by calling **offer** repeatedly is $O(n \log n)$ as a simple upper bound. Bottom-up heapify uses the structure more efficiently.

Java PriorityQueue contracts that matter

- It is a min-heap under natural order unless a comparator changes priority.
- `peek` reads the head; `poll` removes it; both return `null` on an empty queue.
- Iteration order is unspecified and is not sorted order.
- It does not efficiently remove an arbitrary known item; that search can be linear.
- Mutating fields used by an enqueued object's comparator does not automatically reposition it.
- It is not thread-safe.

For a max-heap of integers, use `Comparator.reverseOrder()`. For custom records, compare with `Comparator.comparingInt` or `Integer.compare`, never subtraction:

JAVA

```
Comparator<Job> byPriority = Comparator.comparingInt(Job::priority);
```

`first.priority() - second.priority()` can overflow and reverse ordering.

Heap or quickselect for kth largest?

Workload	Technique	Time	Space	Important property
one kth query, fixed unsorted array	randomized quickselect	expected $O(n)$	$O(1)$ if mutation allowed	input order is destroyed
one kth query, preserve input	quickselect on a clone	expected $O(n)$	$O(n)$ clone	clear ownership contract
stream with unknown future values	min-heap of size k	$O(n \log k)$	$O(k)$	answer updates online
many ranks or sorted result needed	sort	$O(n \log n)$	implementation-dependent	establishes complete order
merge sorted streams	heap of current heads	$O(N \log m)$	$O(m)$	one frontier item per stream

Do not choose a heap merely because the prompt says "largest." The operation pattern decides.

Iterative randomized three-way quickselect

For one-based kth largest, convert to the zero-based ascending target:

TEXT

```
target = length - k
```

Partition the active region into $< \text{pivot}$, $= \text{pivot}$, and $> \text{pivot}$:

TEXT

```
[ values < p | values == p | unknown | values > p ]  
      ^ target inside this equal band means answer found
```

After partition:

- target before the equal band: keep the left region;
- target after the equal band: keep the right region;
- target in the band: return the pivot value.

The loop discards at least the equal band each iteration. This is particularly helpful for duplicate-heavy input. Random pivot choice makes consistently bad partitions unlikely, producing expected $O(n)$ time, but the formal worst case remains $O(n^2)$.

The companion clones caller input and accepts a seed so randomized tests are reproducible. In a space-constrained interview, ask whether mutation is allowed and remove the clone if it is.

Top-k design choices beyond the toy problem

For a streaming top-k service, clarify:

- Is k fixed or changed by clients?
- Are duplicate values separate events?
- Must ties have deterministic order?
- Is deletion or correction required?
- Is the result exact or can it be approximate?
- Does one process hold the stream, or must partial top-k sets merge?

A size- k min-heap retains the largest k values seen; its root is the kth largest only after at least k values arrive. If k changes upward, discarded history cannot be recovered without another store. A good SDE-2 answer calls out that product contract.

Edge-case matrix

Case	Expected handling	Frequent failure
empty heap	<code>peek/poll</code> follow explicit failure contract	reading backing index zero
first resize	grow from nonzero capacity	doubling zero to zero
duplicate priorities	retain all unless dedup is specified	treating heap as a set
integer extremes	direct comparisons or safe comparator	subtraction overflow
one child during sift-down	compare only existing child	reading past logical size
stale object priority	remove/reinsert or use immutable priority	assuming automatic repositioning
heap iteration	unordered except head guarantee	presenting iterator output as sorted
$k = 1$	maximum for kth-largest contract	off-by-one target conversion
$k = n$	minimum	rejecting a valid boundary rank
invalid k or empty input	throw with explicit contract	returning an ambiguous sentinel
all values equal	equal band ends selection immediately	repeatedly partitioning one element
caller input ownership	clone or document mutation	surprising destructive selection

Real interview follow-up round

Interviewer: Why is a heap not a BST?

Candidate: A heap guarantees only parent-child priority and optimizes access to one extreme. It cannot efficiently search for an arbitrary value. A BST maintains left/right key order and supports ordered search and traversal, with different shape requirements.

Interviewer: Why is bottom-up heapify linear?

Candidate: The $O(\log n)$ maximum applies only near the root. Most nodes are leaves or one level above them. Summing the amount of possible movement over all node heights gives a convergent weighted series bounded by $O(n)$.

Interviewer: Your priority queue contains objects. I changed one object's priority. What happens?

Candidate: The queue is not notified, so its internal heap can be inconsistent with current comparator values. Priority fields should be immutable while enqueued, or the item must be removed and reinserted through a supported design.

Interviewer: Why quickselect instead of a size-k heap?

Candidate: For one query over an in-memory array, quickselect is expected linear and uses no ongoing heap. For a stream or when values arrive incrementally, quickselect cannot maintain the answer, while a size-k heap does so in $O(\log k)$ per accepted value.

Interviewer: How do you know your heap code works?

Candidate: After every random offer or poll, I verify parent order, size, and head against Java's `PriorityQueue`, then drain both and compare removal order. I compare quickselect at every rank with a sorted clone. Those differential tests are in the companion.

Run the verified companion

BASH

```
javac -Xlint:all -Werror HeapInterviewChecks.java
java HeapInterviewChecks
```

Expected final line:

TEXT

PASS 17 heap checks

Continue to the graph volume when a heap is used as a frontier for Dijkstra or Prim. Remember that the heap supplies the next candidate; the graph invariant is what makes choosing that candidate correct.

SDE-2 CORE CHAPTER

Chapter 6 - Indexed Heaps, Decrease-Key, and Lazy Deletion

Why this chapter exists

`PriorityQueue` in Java supports `add`, `peek`, and `poll`. It does **not** support "the priority of an element already in the queue has changed, update its position" - and that operation, **decrease-key**, is exactly what Dijkstra's algorithm and Prim's algorithm are written in terms of in every textbook.

That gap is a real interview topic and a real engineering one. There are two standard resolutions, they have different complexities, and knowing which one the JDK forces you into is a question candidates routinely fumble.

The chapter also covers the other operation `PriorityQueue` handles badly: removing an arbitrary element, which is $O(n)$ because it must scan to find it.

The problem, precisely

A binary heap maintains the heap property by sifting elements up or down. Both are $O(\log n)$ **given the element's index in the backing array**. The difficulty is not the sift; it is that a plain heap offers no way to find an element's current index.

TEXT

<code>poll()</code>	$O(\log n)$	position is known: it is the root
<code>add()</code>	$O(\log n)$	position is known: it is the end
<code>decrease-key</code>	$O(n)$	must SEARCH for the element first
<code>remove(x)</code>	$O(n)$	same problem

Java's `PriorityQueue.remove(Object)` is $O(n)$ for exactly this reason - it does a linear scan, then sifts. `contains` is $O(n)$ too. Neither is a defect; a heap simply is not an index.

Resolution 1: the indexed heap

Maintain a side map from element to its current array position, and update that map on **every** swap the heap performs.

JAVA (continued 1/3)

```
import java.util.HashMap;
import java.util.Map;

/** A min-heap that supports decrease-key and arbitrary removal in  $O(\log n)$ . */
public final class IndexedMinHeap<K> {
    private final Object[] elements;
    private final int[] priority;
    private final Map<K, Integer> position = new HashMap<>();
    private int size;

    public IndexedMinHeap(int capacity) {
        elements = new Object[capacity];
        priority = new int[capacity];
    }

    public boolean contains(K key) {
        return position.containsKey(key);    //  $O(1)$ , unlike PriorityQueue
    }

    public void add(K key, int keyPriority) {
        if (position.containsKey(key)) {
            throw new IllegalArgumentException("duplicate key: " + key);
        }
        elements[size] = key;
        priority[size] = keyPriority;
        position.put(key, size);
        siftUp(size++);
    }

    /** Lower an existing key's priority.  $O(\log n)$  because position is known. */
    public void decreaseKey(K key, int newPriority) {
        Integer index = position.get(key);
        if (index == null) {
            throw new IllegalArgumentException("absent key: " + key);
        }
    }
}
```

JAVA (continued 2/3)

```
        if (newPriority > priority[index]) {
            throw new IllegalArgumentException("decreaseKey must not raise priority");
        }
        priority[index] = newPriority;
        siftUp(index);                // only ever moves up
    }

    @SuppressWarnings("unchecked")
    public K poll() {
        if (size == 0) {
            throw new IllegalStateException("empty heap");
        }
        K best = (K) elements[0];
        swap(0, --size);
        position.remove(best);
        elements[size] = null;        // release the reference
        if (size > 0) {
            siftDown(0);
        }
        return best;
    }

    private void siftUp(int index) {
        while (index > 0) {
            int parent = (index - 1) / 2;
            if (priority[parent] <= priority[index]) {
                break;
            }
            swap(parent, index);
            index = parent;
        }
    }

    private void siftDown(int index) {
        while (true) {
```

JAVA (continued 3/3)

```

        int left = 2 * index + 1;
        if (left >= size) {
            return;
        }
        int smallest = left;
        int right = left + 1;
        if (right < size && priority[right] < priority[left]) {
            smallest = right;
        }
        if (priority[index] <= priority[smallest]) {
            return;
        }
        swap(index, smallest);
        index = smallest;
    }
}

/** Every positional change goes through here, which is what keeps the map true. */
@SuppressWarnings("unchecked")
private void swap(int a, int b) {
    Object elementA = elements[a];
    Object elementB = elements[b];
    elements[a] = elementB;
    elements[b] = elementA;
    int priorityA = priority[a];
    priority[a] = priority[b];
    priority[b] = priorityA;
    position.put((K) elementB, a);
    position.put((K) elementA, b);
}

public int size() {
    return size;
}
}

```

The design rule is the whole technique: **every positional change goes through one `swap` method, and that method updates the map**. Scatter the index arithmetic across `siftUp` and `siftDown` and the map will drift out of sync - producing a heap that silently returns wrong elements, with no exception to point at the cause.

`decreaseKey` only ever sifts *up*, because lowering a priority in a min-heap can only move an element closer to the root. A general `changeKey` would sift up or down depending on direction; restricting to decrease keeps the contract explicit and catches the misuse.

Resolution 2: lazy deletion

The indexed heap is the right structure, and in an interview it is often more code than the time allows. The lazy alternative is what most competitive and production Dijkstra implementations actually do.

Never update anything. Push a new entry with the better priority and ignore stale entries when they surface.

JAVA

```
record Entry(int node, int distance) {}

static int[] dijkstra(List<List<int[]>> adjacency, int source) {
    int n = adjacency.size();
    int[] best = new int[n];
    Arrays.fill(best, Integer.MAX_VALUE);
    best[source] = 0;

    PriorityQueue<Entry> queue =
        new PriorityQueue<>(Comparator.comparingInt(Entry::distance));
    queue.add(new Entry(source, 0));

    while (!queue.isEmpty()) {
        Entry current = queue.poll();
        if (current.distance() > best[current.node()]) {
            continue; // stale: a better entry was processed already
        }
        for (int[] edge : adjacency.get(current.node())) {
            int next = edge[0];
            int weight = edge[1];
            long candidate = (long) current.distance() + weight; // avoid overflow
            if (candidate < best[next]) {
                best[next] = (int) candidate;
                queue.add(new Entry(next, best[next])); // push, do not update
            }
        }
    }
    return best;
}
```

The `current.distance() > best[current.node()]` check is the entire mechanism. A node may sit in the queue several times at different distances; the first time it is polled it carries the smallest, and every later copy is discarded in $O(1)$.

The trade: the queue can hold up to E entries rather than V , so complexity is $O(E \log E)$ instead of $O(E \log V)$. Since $E \leq V^2$, $\log E \leq 2 \log V$, so the two differ by at most a constant factor. Memory is genuinely higher.

	Indexed heap	Lazy deletion
Complexity	$O(E \log V)$	$O(E \log E)$ - same to a constant
Queue size	at most V	up to E
Code	~80 lines	~15 lines
Correctness risk	map/heap desync	none; the relaxation guard carries it
Interview default	when asked for it	yes

Lazy deletion is the right default answer, and the right *complete* answer names the memory cost and says the indexed heap is what removes it. Presenting lazy deletion without acknowledging the larger queue is the incomplete version.

Omitting the staleness check is a **performance** bug, not a correctness one, and it is worth being precise about why. A stale entry carries a distance no smaller than the best known, so relaxing from it can only propose values no better than what the guard `candidate < best[next]` already rejects. The answers stay correct; the work does not. Measured over 300 random graphs, dropping the check cost about 17% more edge relaxations, and the penalty grows with graph density.

Candidates often assert that omitting it breaks correctness. It does not, and claiming so invites an interviewer to ask for the failing input - which does not exist for this formulation. The accurate answer is that the relaxation guard carries correctness and the staleness check carries efficiency.

Removing an arbitrary element

The same two options apply.

- **Indexed heap:** look up the position, swap with the last element, shrink, then sift the moved element *either* up or down - it can go either way, unlike `poll` where it can only sink. Forgetting the up case is a classic bug that leaves a heap violating its own invariant.
- **Lazy:** keep a `Set` of removed keys and skip them on `poll`. Simple, but the heap holds garbage until it surfaces, and a workload that removes most of what it inserts wastes proportional memory.

Java's `PriorityQueue.remove(Object)` takes neither approach - it scans linearly. If your algorithm removes arbitrary elements frequently, `PriorityQueue` is the wrong structure and saying so is the answer.

When a heap is the wrong structure

Worth stating plainly, because "use a heap" is over-applied.

- **You need the k-th element once, not a stream.** Quickselect is $O(n)$ average versus $O(n \log k)$ for a heap, and the internals chapter covers it.
- **You need ordered iteration or range queries.** A heap gives you the minimum and nothing else; `TreeMap` or a balanced BST gives order.
- **You need both ends.** A binary heap is one-directional. Use two heaps, a `TreeMap`, or an interval heap.
- **Priorities change constantly and you need exact ordering.** Every change is a sift; a different structure may fit better.
- **Everything is available up front and you want it all sorted.** Sorting is $O(n \log n)$ with better constants and cache behaviour than n polls.

The sliding-window-maximum problem is the sharpest example: a heap gives $O(n \log n)$ with lazy deletion, while a monotonic deque gives $O(n)$. The ordering-structures volume covers the deque; the point here is recognizing that reaching for a heap is not automatically right.

WATCH OUT | Edge cases and common mistakes

- Expecting `PriorityQueue` to support decrease-key. It does not.
- Assuming `PriorityQueue.remove(Object)` or `contains` is better than $O(n)$.
- Scattering index updates instead of funnelling every move through one `swap`.
- Letting `decreaseKey` raise a priority, which requires sifting down and breaks the contract.
- Omitting the staleness check in lazy Dijkstra, wasting roughly 17% of relaxations on stale entries.
- Claiming that omission breaks correctness; the relaxation guard still prevents a worse value from being written.
- Adding a duplicate key to an indexed heap, corrupting the position map.
- Forgetting that arbitrary removal may sift up *or* down, unlike `poll`.
- Not nulling the vacated slot after `poll`, retaining a reference the caller expects released.
- Overflowing on `distance + weight` with large weights; accumulate in `long`.
- Claiming lazy deletion has the same memory profile as an indexed heap.
- Using a heap for a one-shot k-th element where quickselect is $O(n)$.
- Using a heap for sliding window maximum where a monotonic deque is $O(n)$.

TRY IT | Interview questions and model answers

Java's `PriorityQueue` has no decrease-key. How do you write Dijkstra?

Lazy deletion: never update an entry, push a new one with the improved distance, and discard stale entries on `poll` by checking whether the popped distance exceeds the best known for that node. That check is what makes it correct. The queue can hold up to E entries rather than V , so it is $O(E \log E)$ rather than $O(E \log V)$ - the same to within a constant, at higher memory.

When would you build an indexed heap instead?

When queue memory matters, or the algorithm genuinely needs `contains` and arbitrary removal in better than $O(n)$. It maps each element to its current array index and updates that map on every swap, giving $O(\log n)$ decrease-key and removal. The discipline is that all positional changes funnel through one swap method; scattering them lets the map drift and the heap silently returns wrong elements.

Why does `decreaseKey` only sift up?

Lowering a value in a min-heap can only move it closer to the root, never away, so only the upward direction can be violated. A general priority change would need both directions, which is why restricting the operation to decrease keeps the contract clear and catches misuse.

What happens if you drop the staleness check?

You waste work, not correctness. A stale entry's distance is no smaller than the best known, so anything it proposes is rejected by the `candidate < best[next]` guard - that guard is what carries correctness. Measured on random graphs, dropping the check cost about 17% more edge relaxations. It is worth being exact here, because asserting it breaks correctness invites a request for the failing input, and there is not one.

What is `PriorityQueue.remove(Object)` complexity?

$O(n)$, because it scans linearly to find the element before sifting. `contains` is also $O(n)$. A heap is not an index. If your algorithm removes arbitrary elements often, use an indexed heap or lazy deletion with a removed-set.

When is a heap the wrong tool?

For a one-shot k -th element, where quickselect is $O(n)$ rather than $O(n \log k)$. For ordered iteration or range queries, which need a `TreeMap`. For access to both extremes. And for sliding window maximum, where a monotonic deque is $O(n)$ against the heap's $O(n \log n)$.

TRY IT | Exercises

- 1 **Foundation:** Trace the array and position map through three `add` calls and one `poll` on an indexed heap of capacity 8.
- 2 **Foundation:** Time `PriorityQueue.remove(Object)` at sizes 10^3 , 10^4 , and 10^5 and confirm the growth is linear.
- 3 **Interview Core:** Implement the indexed heap above and assert after every operation that the position map matches the array.
- 4 **Interview Core:** Move the position updates out of `swap` into `siftUp` and `siftDown`, then find the sequence where the map desynchronizes.
- 5 **Interview Core:** Implement lazy Dijkstra. Remove the staleness check, confirm over random graphs that the answers stay correct, and measure the extra edge relaxations. Then say which line actually carries correctness.
- 6 **Interview Core:** Instrument both Dijkstras to report peak queue size on a dense graph and compare against V and E .
- 7 **SDE-2 Follow-up:** Implement arbitrary removal on the indexed heap. Sift only downward, then construct the input that leaves the heap invalid.
- 8 **SDE-2 Follow-up:** Solve sliding window maximum with a heap and with a monotonic deque; compare runtimes at 10^6 elements.
- 9 **SDE-2 Follow-up:** Add `decreaseKey` misuse - raising a priority - and show the heap property it breaks.
- 10 **Challenge:** Implement Prim's algorithm with both approaches and state which you would present in an interview and why.

RECAP | Chapter summary

`PriorityQueue` gives you `add`, `peek`, and `poll` in $O(\log n)$ and nothing else in better than $O(n)$, because a heap is not an index - it cannot find an element it already holds. Two resolutions exist. The indexed heap maintains a map from element to current array position, updated on every swap, giving $O(\log n)$ decrease-key, $O(1)$ `contains`, and $O(\log n)$ arbitrary removal; the discipline is funnelling all positional change through a single swap method, since a drifting map produces silently wrong results. Lazy deletion never updates anything, pushes a fresh entry at the better priority, and discards stale entries on poll with one comparison - fifteen lines instead of eighty, at the cost of a queue holding up to E rather than V entries. Lazy is the right interview default provided you name that memory cost. Note precisely what the staleness check does: correctness is carried by the relaxation guard, and the check buys efficiency, worth roughly 17% of relaxations on random graphs. And a heap is not always the answer: quickselect beats it for a one-shot selection, a `TreeMap` beats it for ordered access, and a monotonic deque beats it for sliding window maximum.

TRY IT | Revision checklist

- ☐ I know `PriorityQueue` has no decrease-key and that `remove` and `contains` are $O(n)$.
- ☐ I can explain why a heap cannot locate an element it holds.
- ☐ I can implement an indexed heap and state the single-swap discipline.
- ☐ I know `decreaseKey` sifts only up, and why.
- ☐ I can write lazy Dijkstra and explain the staleness check.
- ☐ I know omitting that check costs work, not correctness, and which line carries correctness.
- ☐ I can state the queue-size and complexity difference between the two approaches.
- ☐ I know arbitrary removal may sift either direction.
- ☐ I accumulate path distances in `long` to avoid overflow.
- ☐ I can name three problems where a heap is the wrong structure.

SDE-2 CORE CHAPTER

Chapter 7 - Heaps and Priority Queues Practice Lab

- 1 Draw the array indexes for a seven-node complete binary tree.
- 2 State the min-heap invariant and explain why index 0 is minimal.
- 3 Predict whether iterating a `PriorityQueue` constructed from several values would produce sorted order.
- 4 Debug comparator `(a, b) -> b.score - a.score`.
- 5 Debug a heap of mutable jobs whose priority changes after insertion.
- 6 Implement sift-down and bottom-up heapify.
- 7 Return the k smallest values without sorting all input.
- 8 Return top-k frequent values with deterministic tie-breaking.
- 9 Merge k sorted arrays while preserving source positions.
- 10 Schedule meeting rooms and return the minimum room count.
- 11 Implement a running median with overflow-safe averaging.
- 12 Compare sorting, bounded heap, and quickselect for kth-largest selection.
- 13 Explain lazy deletion and its memory-cleanup obligation.
- 14 Design a stable priority scheduler with equal priorities.
- 15 Explain why arbitrary deletion is a boundary for Java `PriorityQueue`.
- 16 **Interview Core:** Return the k closest points with a bounded max-heap, overflow-safe squared distance, and deterministic tie-breaking.
- 17 **SDE-2 Follow-up:** Find the smallest range containing at least one value from each of k sorted lists. Explain why only the list supplying the current minimum advances.

Heap and selection internals lab

- 18 **Foundation:** Map every parent and child index in heap array `[1, 3, 2, 8, 5, 7]`. Explain why the array is not globally sorted.
- 19 **Interview Core:** Implement a resizable primitive min-heap with bottom-up heapify, offer, peek, poll, and an invariant checker.
- 20 **Interview Core:** Prove bottom-up heapify is $O(n)$ by grouping nodes by height; do not multiply every node by maximum height.
- 21 **Interview Core:** Implement iterative randomized three-way quickselect for a one-based kth-largest contract. Test duplicates, $k=1$, and $k=n$.
- 22 **SDE-2 Follow-up:** Compare custom heap behavior with `PriorityQueue` after every operation in a deterministic randomized sequence, then drain both.
- 23 **SDE-2 Follow-up:** Design a top-k stream contract when k may increase after values were discarded. State what history must be retained or why the request cannot be answered exactly.

SDE-2 CORE CHAPTER

Chapter 8 - Heaps and Priority Queues Practice Lab Solutions

- 1 Children of index i are $2i+1$ and $2i+2$; parent is $(i-1)/2$ for nonroot indexes.
- 2 Every parent is no greater than its children. Repeated parent relationships imply the root is no greater than any descendant.
- 3 No. Iteration exposes heap-array order; only the head is guaranteed minimal. Poll a copy repeatedly for sorted values.
- 4 Subtraction can overflow and reverse ordering. Use `Comparator.comparingInt(Job::score).reversed()` plus a deterministic tie-breaker.
- 5 The queue does not repair its position. Remove and reinsert through a controlled update API, or store immutable priority snapshots and discard stale entries lazily.
- 6 Compare a node with its smaller child and swap until valid. Heapify from the last parent down; aggregate height work is $O(n)$.
- 7 Maintain a max-heap of size k ; its root is the weakest retained small value. $O(n \log k)$ time, $O(k)$ space.
- 8 Count frequencies, then use a bounded heap or sort entries. Define whether equal counts use numeric, lexical, or first-seen order.
- 9 Heap entries contain `(value, source, index)`. Poll, emit, then enqueue the next index from that source. $O(N \log k)$.
- 10 Sort intervals by start, keep end times in a min-heap, reuse all rooms ending before the next start, and track peak heap size. Clarify whether touching endpoints overlap.
- 11 Use lower max-heap and upper min-heap. Average as `((long)a + b) / 2.0`.
- 12 Sorting is $O(n \log n)$ and orders everything; bounded heap is $O(n \log k)$, streaming, and $O(k)$ space; quickselect is expected $O(n)$, mutates or copies input, and has $O(n^2)$ worst case without stronger pivot guarantees.
- 13 Stale heap entries remain until they reach the top. Track validity/version counts and periodically compact if stale memory can grow too large.
- 14 Compare priority first and monotonic sequence number second. Sequence generation, overflow, and concurrency must be controlled.
- 15 `remove(object)` generally searches linearly before repairing the heap. Use an indexed heap or lazy invalidation when arbitrary updates/removals are core operations.

- 16 Keep at most k points in a max-heap whose root is the worst retained candidate. Compute squared distance in `long`, compare with `Long.compare`, and evict after each insertion beyond k . Sorting the retained k values is necessary only when the output contract requires order. Time is $O(n \log k)$, space $O(k)$.
- 17 Seed a min-heap with one entry per list and track the frontier maximum. The heap minimum and maximum define a complete range. Advancing a nonminimum list cannot improve the left boundary, so advance the minimum's source. Stop when that source ends because no later frontier covers every list. Total time is $O(N \log k)$, space $O(k)$.

Heap and selection internals solutions

- 18 Index 0 has children 1 and 2; index 1 has 3 and 4; index 2 has 5 and 6, although 6 is absent in the six-value example. Nonroot parent is $(i - 1) / 2$. Parent order proves only that each ancestor is no greater than descendants; siblings and cousins need not be ordered, so level-order array storage is not sorted output.
- 19 Append and sift up for offer; for poll, save root, move the last logical value to root, shrink size, and sift down through the smaller child. Grow a nonzero backing array geometrically. Bottom-up construction starts at $size/2 - 1$. After operations, verify every `heap[(i-1)/2] <= heap[i]`. `IntMinHeap` provides the complete primitive implementation and extreme-value tests.
- 20 Leaves have zero sift distance. At most $n/4$ nodes have height at least one, $n/8$ height at least two, and so forth. Total work is bounded by $n/4 * 1 + n/8 * 2 + \dots$, a convergent series in $O(n)$. Charging every node $\log n$ ignores the complete tree's height distribution.
- 21 Convert one-based k th largest to ascending target $n - k$. In each active range, choose a random pivot and form less/equal/greater regions. Continue only left or right when the target lies outside the equal band; return pivot when it lies inside. Validate $1 \leq k \leq n$. The companion clones input, making preservation explicit, and handles all-equal input in one partition.
- 22 Maintain custom and standard min-heaps. With a fixed seed, mostly offer random integers and sometimes poll when nonempty. After every step compare size, root, and custom invariant. Drain both and compare all remaining outputs. Include `Integer.MIN_VALUE` and `MAX_VALUE` targeted tests because ordinary randomness rarely samples them.
- 23 A bounded heap discards values outside the current top k . If k later increases, those values may now belong to the answer but are gone. Exact support requires retaining the full stream, durable ranked partitions, or a bounded maximum k chosen in advance. Otherwise the API must reject upward changes or promise only approximate/history-limited results. This is a contract constraint, not a different heap trick.

SDE-2 CORE CHAPTER

Chapter 9 - Executable Heap and Selection Checks

This dependency-free Java 21 companion validates a resizable primitive binary heap, bottom-up heapify, sift invariants, bounded top-k, running median, randomized three-way quickselect, and differential cases. A successful run prints PASS 17 heap checks.

JAVA (continued 1/9)

```
import java.util.Arrays;
import java.util.Comparator;
import java.util.NoSuchElementException;
import java.util.OptionalInt;
import java.util.PriorityQueue;
import java.util.Random;

public final class HeapInterviewChecks {
    private HeapInterviewChecks() {}

    /** A resizable primitive min-heap: parent=(i-1)/2, children=2i+1 and 2i+2. */
    static final class IntMinHeap {
        private int[] heap;
        private int size;

        IntMinHeap() {
            heap = new int[1];
        }

        IntMinHeap(int[] values) {
            heap = Arrays.copyOf(values, Math.max(1, values.length));
            size = values.length;
            for (int parent = size / 2 - 1; parent >= 0; parent--) {
                siftDown(parent);
            }
        }

        int size() {
            return size;
        }

        int capacity() {
            return heap.length;
        }
    }
}
```

JAVA (continued 2/9)

```
boolean isEmpty() {
    return size == 0;
}

void offer(int value) {
    ensureCapacity();
    heap[size] = value;
    siftUp(size++);
}

int peek() {
    if (size == 0) {
        throw new NoSuchElementException("empty heap");
    }
    return heap[0];
}

int poll() {
    int minimum = peek();
    heap[0] = heap[--size];
    if (size > 0) {
        siftDown(0);
    }
    return minimum;
}

boolean invariantHolds() {
    for (int child = 1; child < size; child++) {
        int parent = (child - 1) / 2;
        if (heap[parent] > heap[child]) {
            return false;
        }
    }
}
```

JAVA (continued 3/9)

```
        return true;
    }

    int[] levelOrderForTeaching() {
        return Arrays.copyOf(heap, size);
    }

    private void siftUp(int child) {
        while (child > 0) {
            int parent = (child - 1) / 2;
            if (heap[parent] <= heap[child]) {
                return;
            }
            swap(heap, parent, child);
            child = parent;
        }
    }

    private void siftDown(int parent) {
        while (true) {
            int left = parent * 2 + 1;
            if (left >= size) {
                return;
            }
            int right = left + 1;
            int smaller = right < size && heap[right] < heap[left] ? right : left;
            if (heap[parent] <= heap[smaller]) {
                return;
            }
            swap(heap, parent, smaller);
            parent = smaller;
        }
    }
}
```

JAVA (continued 4/9)

```
private void ensureCapacity() {
    if (size < heap.length) {
        return;
    }
    int newCapacity = heap.length <= Integer.MAX_VALUE / 2
        ? heap.length * 2 : Integer.MAX_VALUE;
    if (newCapacity == heap.length) {
        throw new IllegalStateException("heap is too large");
    }
    heap = Arrays.copyOf(heap, newCapacity);
}

static final class KthLargest {
    private final int k;
    private final PriorityQueue<Integer> largest = new PriorityQueue<>();

    KthLargest(int k) {
        if (k <= 0) {
            throw new IllegalArgumentException("k must be positive");
        }
        this.k = k;
    }

    OptionalInt add(int value) {
        if (largest.size() < k) {
            largest.add(value);
        } else if (value > largest.peek()) {
            largest.poll();
            largest.add(value);
        }
        return largest.size() == k
            ? OptionalInt.of(largest.peek()) : OptionalInt.empty();
    }
}
```

JAVA (continued 5/9)

```
    }  
}  
  
static final class MedianTracker {  
    private final PriorityQueue<Integer> lower =  
        new PriorityQueue<>(Comparator.reverseOrder());  
    private final PriorityQueue<Integer> upper = new PriorityQueue<>();  
  
    void add(int value) {  
        if (lower.isEmpty() || value <= lower.peek()) lower.add(value);  
        else upper.add(value);  
        if (lower.size() > upper.size() + 1) upper.add(lower.poll());  
        else if (upper.size() > lower.size()) lower.add(upper.poll());  
    }  
  
    double median() {  
        if (lower.isEmpty()) throw new IllegalStateException("empty");  
        if (lower.size() != upper.size()) return lower.peek();  
        return ((long) lower.peek() + upper.peek()) / 2.0;  
    }  
}  
  
/** Returns the one-based kth-largest value without changing caller input. */  
static int quickselectKthLargest(int[] input, int k, long seed) {  
    if (input == null || input.length == 0) {  
        throw new IllegalArgumentException("non-empty input required");  
    }  
    if (k < 1 || k > input.length) {  
        throw new IllegalArgumentException("k must be in [1,n]");  
    }  
    int[] values = input.clone();  
    int target = values.length - k;  
    int left = 0;
```

JAVA (continued 6/9)

```
int right = values.length - 1;
Random random = new Random(seed);
while (left <= right) {
    int pivot = values[left + random.nextInt(right - left + 1)];
    int lower = left;
    int scan = left;
    int upper = right;
    while (scan <= upper) {
        if (values[scan] < pivot) {
            swap(values, lower++, scan++);
        } else if (values[scan] > pivot) {
            swap(values, scan, upper--);
        } else {
            scan++;
        }
    }
    if (target < lower) {
        right = lower - 1;
    } else if (target > upper) {
        left = upper + 1;
    } else {
        return values[target];
    }
}
throw new AssertionError("valid rank was not found");
}

private static boolean customHeapMatchesPriorityQueue() {
    Random random = new Random(29L);
    IntMinHeap custom = new IntMinHeap();
    PriorityQueue<Integer> standard = new PriorityQueue<>();
    for (int operation = 0; operation < 5_000; operation++) {
        if (standard.isEmpty() || random.nextInt(3) != 0) {
```

JAVA (continued 7/9)

```

        int value = random.nextInt();
        custom.offer(value);
        standard.offer(value);
    } else if (custom.poll() != standard.remove()) {
        return false;
    }
    if (custom.size() != standard.size() || !custom.invariantHolds()
        || (!standard.isEmpty() && custom.peek() != standard.peek())) {
        return false;
    }
}
while (!standard.isEmpty()) {
    if (custom.poll() != standard.remove()) {
        return false;
    }
}
return custom.isEmpty();
}

private static boolean quickselectMatchesSorting() {
    Random random = new Random(31L);
    for (int trial = 0; trial < 1_000; trial++) {
        int length = 1 + random.nextInt(40);
        int[] input = new int[length];
        for (int index = 0; index < length; index++) {
            input[index] = random.nextInt(31) - 15;
        }
        int[] sorted = input.clone();
        Arrays.sort(sorted);
        for (int k = 1; k <= length; k++) {
            int expected = sorted[length - k];
            if (quickselectKthLargest(input, k, trial * 41L + k) != expected) {
                return false;
            }
        }
    }
}

```

JAVA (continued 8/9)

```
        }
    }
}
return true;
}

private static void swap(int[] values, int first, int second) {
    int temporary = values[first];
    values[first] = values[second];
    values[second] = temporary;
}

private static void expectFailure(Runnable action, Class<? extends Throwable> type) {
    try {
        action.run();
    } catch (Throwable failure) {
        if (type.isInstance(failure)) {
            return;
        }
        throw new AssertionError("wrong failure type", failure);
    }
    throw new AssertionError("expected " + type.getSimpleName());
}

private static void check(boolean condition, String message) {
    if (!condition) throw new AssertionError(message);
}

public static void main(String[] args) {
    KthLargest kth = new KthLargest(3);
    check(kth.add(4).isEmpty(), "first");
    check(kth.add(5).isEmpty(), "second");
    check(kth.add(8).orElseThrow() == 4, "third");
}
```

JAVA (continued 9/9)

```

        check(kth.add(2).orElseThrow() == 4, "discard");
        MedianTracker medians = new MedianTracker();
        medians.add(Integer.MAX_VALUE);
        medians.add(Integer.MAX_VALUE);
        check(medians.median() == Integer.MAX_VALUE, "overflow-safe median");

        IntMinHeap heapified = new IntMinHeap(new int[] {7, 2, 9, 2, -1});
        check(heapified.invariantHolds(), "bottom-up heapify invariant");
        check(Arrays.equals(heapified.levelOrderForTeaching(), new int[] {-1, 2, 9, 7, 2}),
            "heap shape is level order, not sorted order");
        check(heapified.poll() == -1 && heapified.poll() == 2 && heapified.poll() == 2,
            "poll order");
        IntMinHeap growing = new IntMinHeap();
        int initialCapacity = growing.capacity();
        growing.offer(Integer.MAX_VALUE);
        growing.offer(Integer.MIN_VALUE);
        check(growing.capacity() > initialCapacity, "backing array resizes");
        check(growing.peek() == Integer.MIN_VALUE, "extreme values need no subtraction comparator");
        expectFailure(() -> new IntMinHeap().poll(), NoSuchElementException.class);

        int[] selectionInput = {5, 1, 5, -2, 9, 9, 3};
        check(quickselectKthLargest(selectionInput, 1, 1L) == 9, "largest rank");
        check(quickselectKthLargest(selectionInput, 4, 2L) == 5, "duplicate rank");
        check(Arrays.equals(selectionInput, new int[] {5, 1, 5, -2, 9, 9, 3}),
            "selection preserves caller input");
        expectFailure(() -> quickselectKthLargest(selectionInput, 0, 3L),
            IllegalArgumentException.class);
        check(customHeapMatchesPriorityQueue(), "heap differential test");
        check(quickselectMatchesSorting(), "selection differential test");

        System.out.println("PASS 17 heap checks");
    }
}

```

Part II - Practice and Handoff

TRY IT | Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: heap operations and kth values
- Interview Core: top-k and k-way merge
- SDE-2 Follow-up: comparator and memory trade-offs
- Challenge: streaming median or scheduler design

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

[Previous book: DSA 13 - Trees, Binary Search Trees, and Tries](#)

[Series index](#)

[Next book: DSA 15 - Graphs: Traversal, Ordering, Paths, and Union-Find](#)

TRY IT | Completion check

- ☐ I can recognize the core patterns without being told the data structure or algorithm.
- ☐ I can implement the representative Java templates from memory.
- ☐ I can explain why each implementation is correct.
- ☐ I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- ☐ I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that segment in order. The same series codes and positions appear on the website, PDF covers, and canonical download folders.

SEGMENT	BOOK	FOCUSED LEARNING PATH
Java	JAVA 01	Java Foundations for Problem Solving
Java	JAVA 02	Git and GitHub for Java Engineers
Java	JAVA 03	Maven and Gradle for Java Engineers
Java	JAVA 04	Language, OOP, and Modern Java
Java	JAVA 05	Collections, Streams, and I/O
Java	JAVA 06	JVM and Java Execution
Java	JAVA 07	Java Concurrency and the Memory Model
Java	JAVA 08	Performance, Diagnostics, and GC Incidents
Java	JAVA 09	Java Question Bank, Study Plan, and Reference
DSA	DSA 01	Time and Space Complexity for Java Interviews
DSA	DSA 02	Number Systems and Math Foundations for DSA Interviews
DSA	DSA 03	Number Systems Interview Patterns and Rapid Revision
DSA	DSA 04	Bit Manipulation in Java
DSA	DSA 05	Loop Mastery, Patterns, and Index Calculations
DSA	DSA 06	Arrays and Array Problem-Solving Patterns
DSA	DSA 07	Strings and String Problem-Solving Patterns
DSA	DSA 08	Hashing: Maps, Sets, Frequency, and Prefix State
DSA	DSA 09	Recursion and Backtracking in Java
DSA	DSA 10	Linked Lists: Pointer Reasoning and Mutation
DSA	DSA 11	Stacks, Queues, Deques, and Monotonic Patterns
DSA	DSA 12	Binary Search: Bounds, Answers, and Invariants
DSA	DSA 13	Trees, Binary Search Trees, and Tries
DSA	DSA 14	Heaps, Priority Queues, Selection, and Top-K
DSA	DSA 15	Graphs: Traversal, Ordering, Paths, and Union-Find
DSA	DSA 16	Greedy Algorithms: Recognition, Proofs, and Scheduling
DSA	DSA 17	Dynamic Programming: State, Transitions, and Optimization
Frameworks	FW 01	MySQL for Java Backend Interviews
Frameworks	FW 02	Hibernate and JPA for Java Backend Interviews
Frameworks	FW 03	Spring Framework for Java Engineers
Frameworks	FW 04	Spring Boot for Java Backend Engineers
Frameworks	FW 05	Spring Boot and REST APIs

SEGMENT	BOOK	FOCUSED LEARNING PATH
Frameworks	FW 06	Spring Data for Java Backend Engineers
Frameworks	FW 07	MongoDB for Java Backend Interviews
Frameworks	FW 08	Redis for Java Backend Interviews
Frameworks	FW 09	Persistence, SQL, JPA, and Caching
Frameworks	FW 10	Apache Kafka and Spring Kafka
Frameworks	FW 11	Spring Ecosystem Extensions
Frameworks	FW 12	Spring AI for Java Engineers
System Design	SD 01	Design, Backend, Testing, and Security
System Design	SD 02	Distributed Systems and System Design
System Design	SD 03	Generative AI System Design

All 40 publication editions are available now. Choose Java, DSA, Frameworks, or System Design first, then follow the books inside that shelf in order. Each deep-dive book remains independently printable and available on the web.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer and the author and editor of the Java SDE-2 Interview Preparation Series. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial approach

Vinay sets the learning sequence and publication standard and performs the final review for Java accuracy, evidence, attribution, and release readiness. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: [LinkedIn](#) | [GitHub](#)

Copyright and Publishing Notes

Copyright 2026 Vinay Reddy Kalluri and credited contributors. Open educational edition.

Book prose, exercises, diagrams, and PDFs are licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). Build scripts and source code are licensed under MIT. Individual credit is recorded in AUTHORS.md and Git history.

Repository: <https://github.com/vinayreddykalluri/SDE2-Interview-Handbook>. Attribution does not imply endorsement. Java and related marks are owned by their respective holders.

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Data Structures and Algorithms - Book 14 of 17 - Study Step DSA-14. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.